With Maven :

```
simple-java-app/
├── pom.xml
│
├── src/
│   ├── main/java/com/example/App.java
│   └── test/java/com/example/AppTest.java
```

Pom.xml:

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>simple-java-app</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <dependencies>
        <!-- JUnit -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

App.java:

```java
package com.example;

public class App {
    public static int add(int a, int b) {
```

```
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println("Result: " + add(5, 3));
    }
}
```

AppTest.java:

```
package com.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class AppTest {

    @Test
    void testAdd() {
        assertEquals(8, App.add(5, 3));
    }
}
```

🔹 STEP 6: Run Maven Commands

✅ 1. Compile

```bash
mvn compile
```

👉 Converts `.java → .class`

✅ 2. Run Tests

```bash
mvn test
```

👉 Runs JUnit test

STEP 1: Create Maven Project

👉 In VS Code / manually create structure:

```
simple-java-app/
|— pom.xml
|
 ├— src/
|    ├— main/java/com/example/App.java
|    └— test/java/com/example/AppTest.java
```

🔹 **STEP 2: Write** pom.xml

👉 This is **most important file in Maven**

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>simple-java-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>
    <!-- JUnit -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

🔹 **STEP 3: Write Java Code**

App.java

```
package com.example;

public class App {
  public static int add(int a, int b) {
    return a + b;
  }
```

```
    public static void main(String[] args) {
        System.out.println("Result: " + add(5, 3));
    }
}
```

🔹 **STEP 4: Write JUnit Test**

AppTest.java

```
package com.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class AppTest {

    @Test
    void testAdd() {
        assertEquals(8, App.add(5, 3));
    }
}
```

🔹 **STEP 5: Open Terminal in Project Folder**

👉 IMPORTANT:

```
cd simple-java-app
```

🔹 **STEP 6: Run Maven Commands**

✅ **1. Compile**

```
mvn compile
```

👉 Converts .java → .class

✅ **2. Run Tests**

```
mvn test
```

👉 Runs JUnit test

✅ **3. Create JAR (IMPORTANT)**

```
mvn package
```

👉 Output:

target/simple-java-app-1.0-SNAPSHOT.jar

✅ 4. Full Build (BEST COMMAND)

mvn clean install

👉 Does everything:

clean → compile → test → package → install

🔹 STEP 7: Verify Output

Go to:

target/

You should see:

simple-java-app-1.0-SNAPSHOT.jar

🔥 HOW IT CONNECTS TO JENKINS

In pipeline:

bat '"C:\\apache-maven\\bin\\mvn" clean package'

👉 Jenkins runs Maven automatically ✓

🎯 VERY SIMPLE FLOW (REMEMBER)

Code → mvn compile → mvn test → mvn package → JAR created

WithOut Maven:

STEP 1: Create Project Structure

```
simple-java-app/
|
 ├── src/
|   └── App.java
```

🔹 **STEP 2: Write Java Code**

App.java

```
public class App {
   public static int add(int a, int b) {
      return a + b;
   }

   public static void main(String[] args) {
      System.out.println("Result: " + add(5, 3));
   }
}
```

🔹 **STEP 3: Open CMD in Folder**

```
cd D:\PROJECT\Agile\simple-java-app\src
```

🔹 **STEP 4: Compile Java**

```
javac App.java
```

👉 This creates:

App.class

🔹 **STEP 5: Run Program**

java App

👉 Output:

Result: 8

🔹 **STEP 6: Create JAR File (IMPORTANT)**

Go one level up:

cd ..

Now run:

jar cvf app.jar -C src .

👉 This creates:

app.jar

🔹 **STEP 7: Run JAR**

java -cp app.jar App

🔥 **HOW TO USE IN JENKINS (WITHOUT MAVEN)**

Replace Maven stage with:

```
stage('Build') {
   steps {
      bat 'javac src\\App.java'
   }
}

stage('Run') {
   steps {
      bat 'java -cp src App'
   }
```

```
}

stage('Package') {
    steps {
        bat 'jar cvf app.jar -C src .'
    }
}
```

🔷 DOCKERFILE (WITHOUT MAVEN)

```
FROM eclipse-temurin:17-jdk-jammy

WORKDIR /app

COPY app.jar app.jar

CMD ["java", "-cp", "app.jar", "App"]
```

🎯 DIFFERENCE (IMPORTANT)

Maven	Without Maven
Automatic build	Manual commands
pom.xml	No config file
Easy for big projects	Simple projects only

Html pipeline code:

```
pipeline { agent any environment { DOCKER_IMAGE = "Dhanush/html-demo" } stages { stage('Clone Code') { steps { git branch: 'main', url: 'https://github.com/dhanush.git' } } stage('Build Docker Image') { steps { bat 'docker build -t %DOCKER_IMAGE% .' } } stage('Push Image') { steps { withCredentials([usernamePassword(credentialsId: 'dockerhub', usernameVariable: 'USER', passwordVariable: 'PASS')]) { bat 'docker login -u %USER% -p %PASS%' bat 'docker push %DOCKER_IMAGE%' } } } stage('Deploy to Kubernetes') { steps { withCredentials([file(credentialsId: 'kuberconfig', variable: 'KUBECONFIG')]) { bat ''' set KUBECONFIG=%KUBECONFIG% kubectl apply -f deployment.yaml --validate=false ''' } } } } }
```